Transformer

GROUP 4

Thành viên

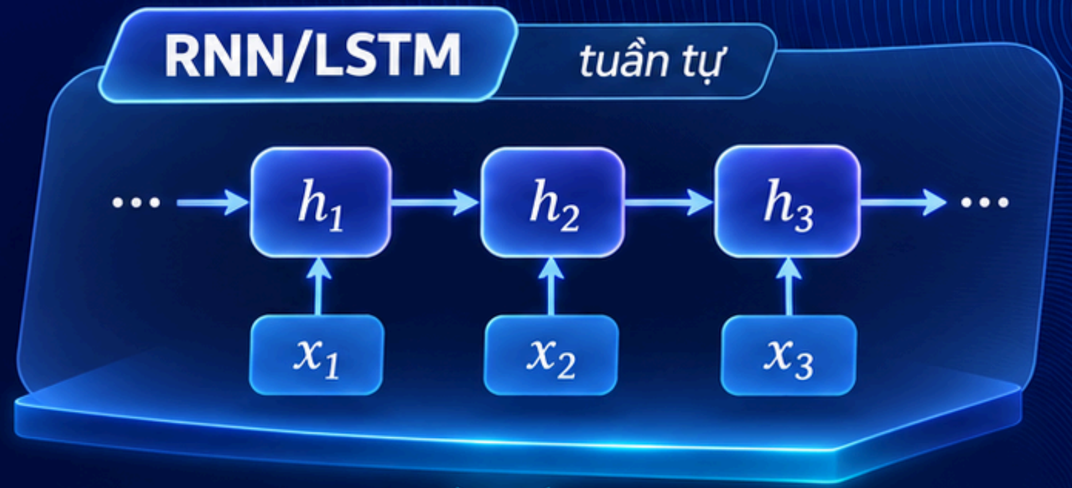
Họ và tên	MSSV	Tỷ lệ hoàn thành
Nguyễn Quốc An	038205000656	100%
Huỳnh Công Xuân Bảo	079206028798	100%
Nguyễn Thị Vân Anh	079306009706	100%
Nguyễn Thanh Bình	089206003339	100%
Nguyễn Hoàng Đăng	079206023549	100%

1. Đặt vấn đề

Vì sao Transformer ra đời?

Hạn chế của RNN/LSTM

- Xử lý tuần tự
- Khó nhớ ngữ cảnh xa
- Khó song song hóa



→ Cần một kiến trúc hiệu quả hơn

2. Giải pháp của Transformer

Self-Attention

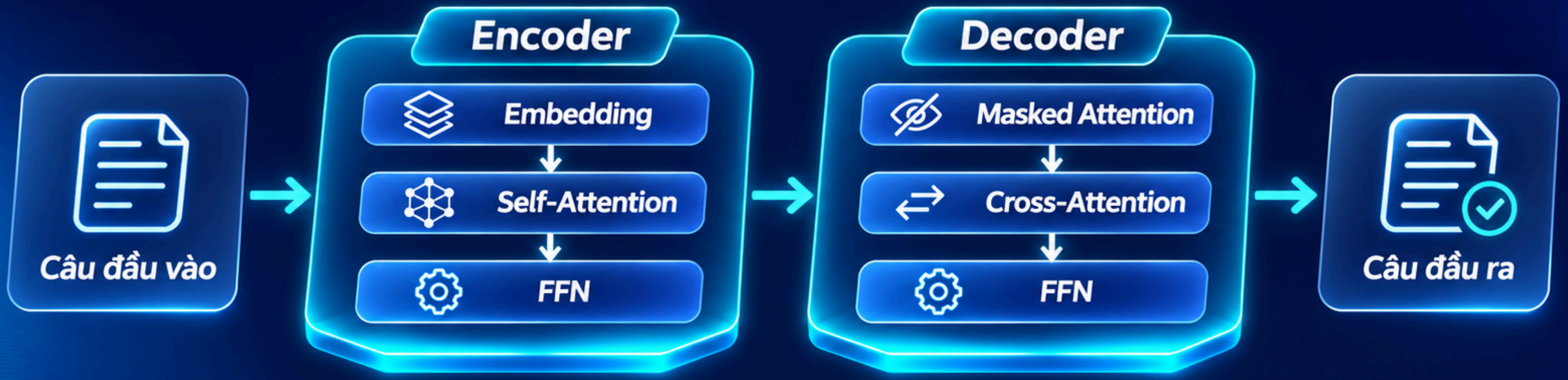
Học quan hệ giữa các từ ở xa nhau

- Không cần đi từng bước
- Liên hệ trực tiếp
- Học ngữ cảnh xa tốt hơn

→ Transformer hiểu phụ thuộc dài hạn tốt hơn

3. Kiến trúc tổng quan
Encoder – Decoder

Luồng dữ liệu trong Transformer



Encoder: hiểu đầu vào

Decoder: tạo đầu ra

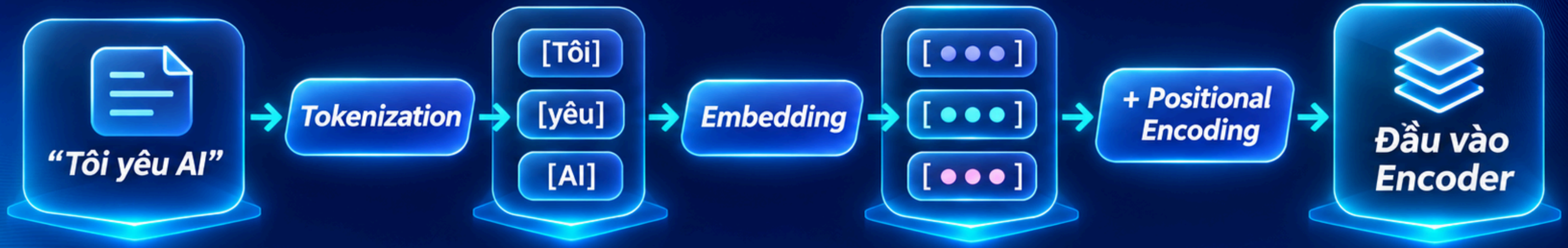
Attention: cơ chế cốt lõi

→ Transformer hoạt động theo kiến trúc Encoder–Decoder

4. Biểu diễn đầu vào

Tokenization, Embedding & Positional Encoding

Từ câu chữ đến vector



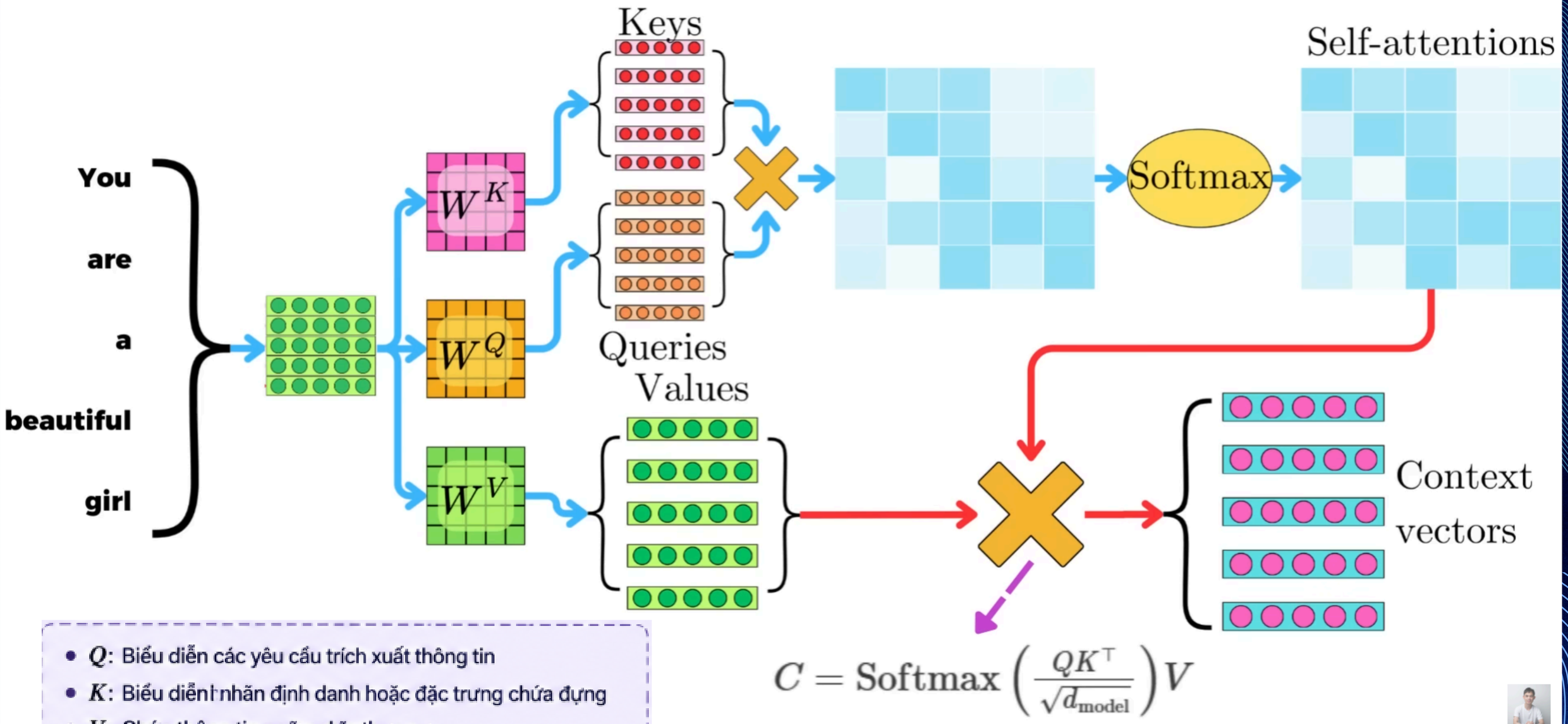
Tách câu thành token

Đổi token thành vector

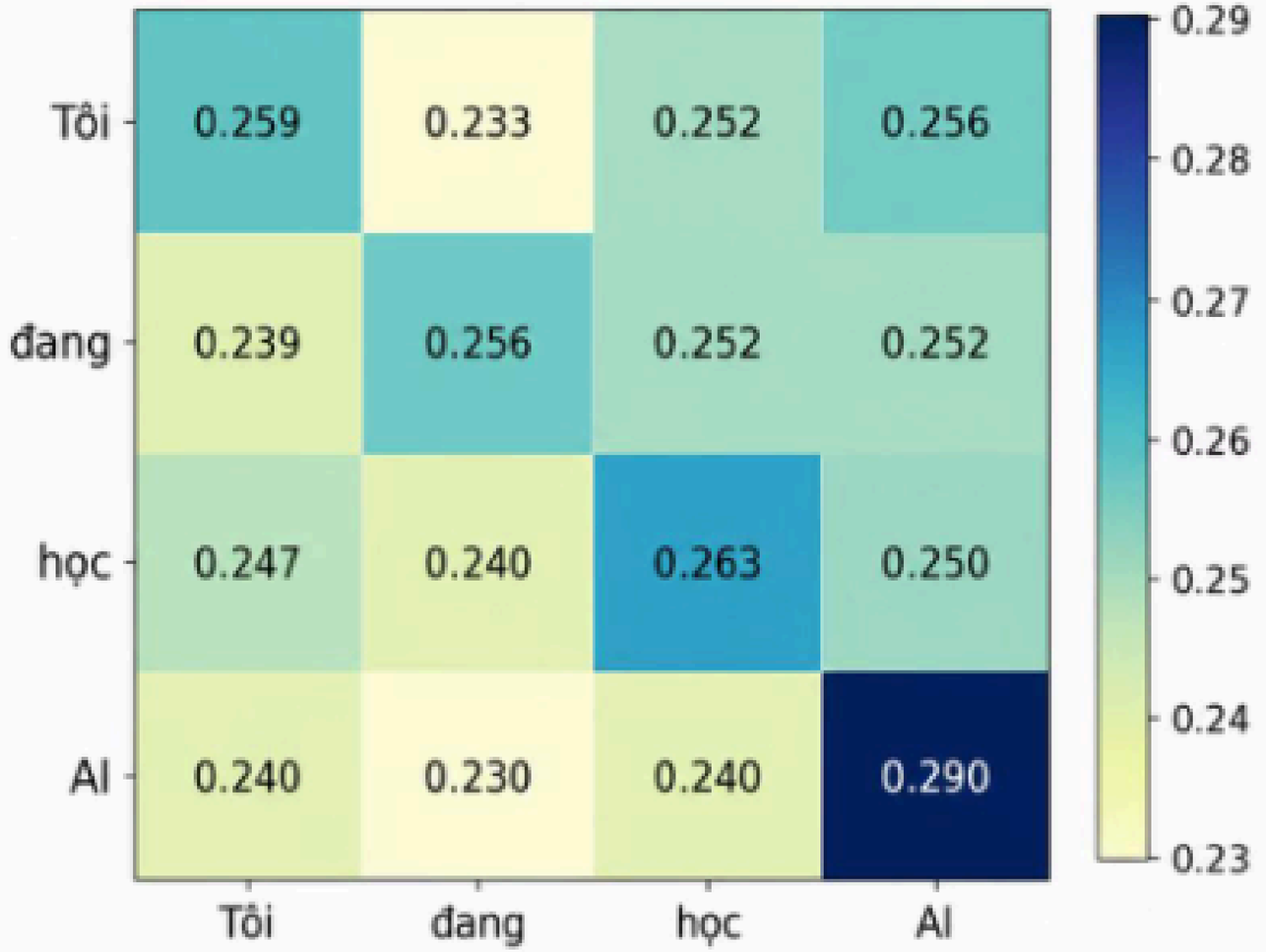
Thêm thông tin vị trí

→ Văn bản phải được biểu diễn thành vector trước khi vào Transformer

self-attention



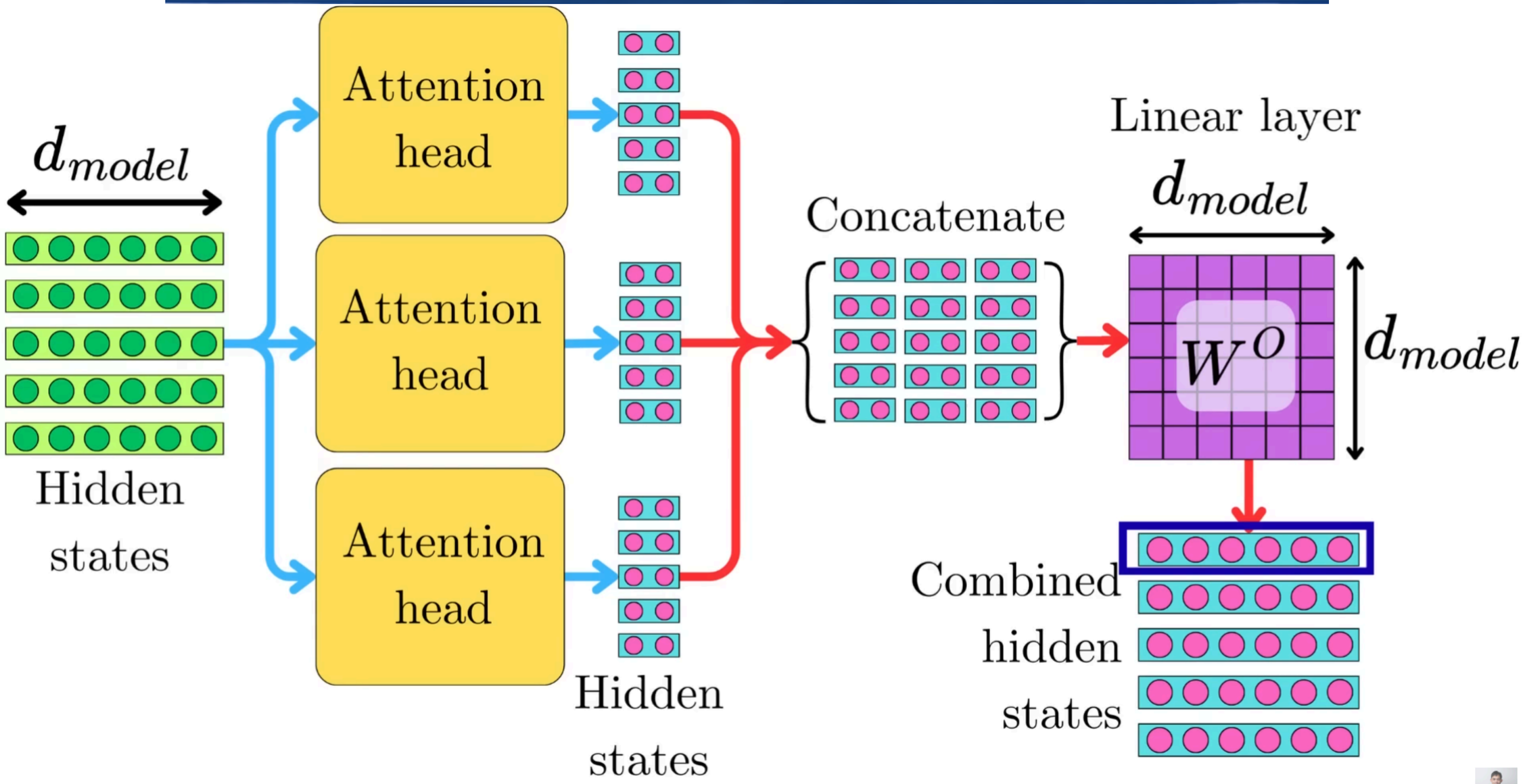
- Q: Biểu diễn các yêu cầu trích xuất thông tin
- K: Biểu diễn nhãn định danh hoặc đặc trưng chứa đựng
- V: Chứa thông tin ngữ nghĩa thực sự



Hình 6.21: Ma trận attention score sau khi đi qua hàm softmax kết hợp với heatmap thể hiện mức độ "chú ý" của một từ lên các từ còn lại trong câu, màu đậm thể hiện trọng số chú ý cao hơn. Cụ thể hơn, hàng của ma trận là các từ truy vấn (Q), cột của ma trận là các từ ngữ cảnh (K).

2. Multi-Head Attention (MHA)

– Nhiều đầu chú ý, nhìn nhận ngữ cảnh đa chiều –

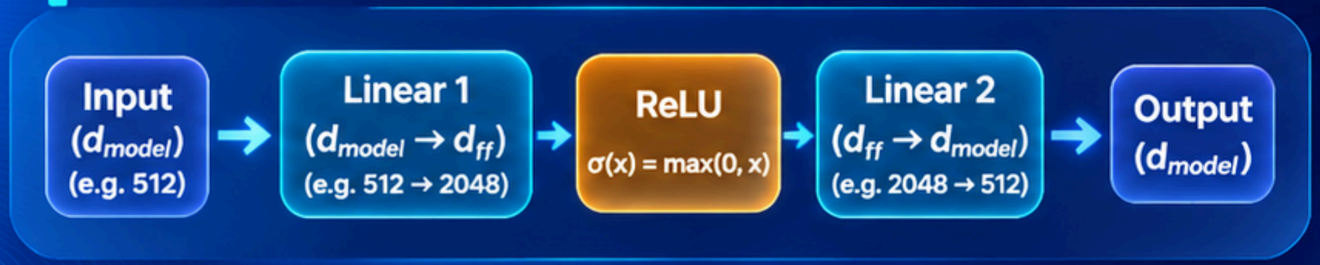


Feed Forward Network (FFN)

FFN là gì?

- Mạng Neural Network truyền thẳng, đặt sau Multi-Head Self-Attention.
- Biến đổi biểu diễn của từng token qua 2 lớp Linear và ReLU.

Cấu trúc

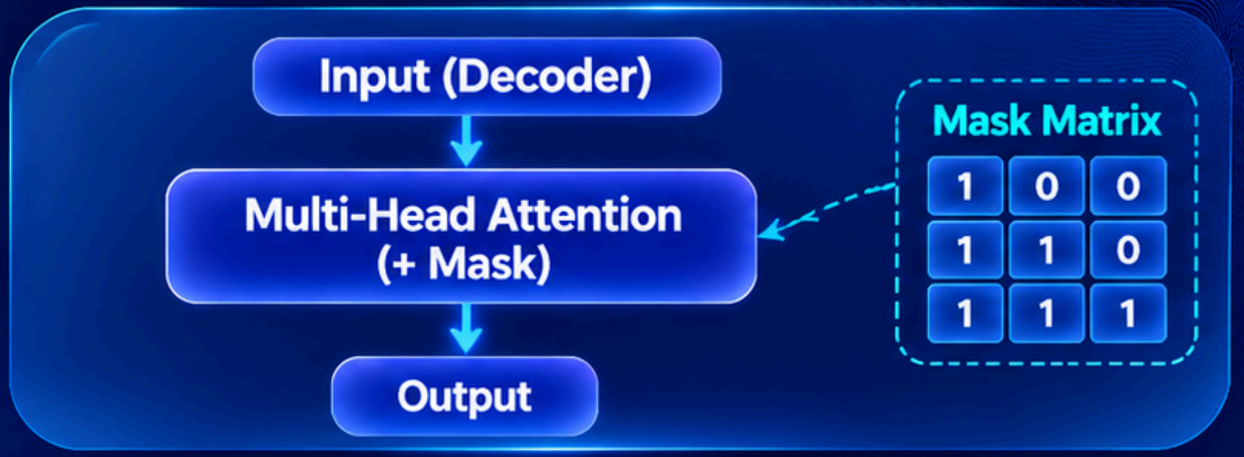


→ **FFN giúp mô hình học các phép biến đổi phi tuyến phức tạp hơn sau khi đã tổng hợp thông tin giữa các token.**

Masked Multi-Head Self-Attention & Cross-Attention

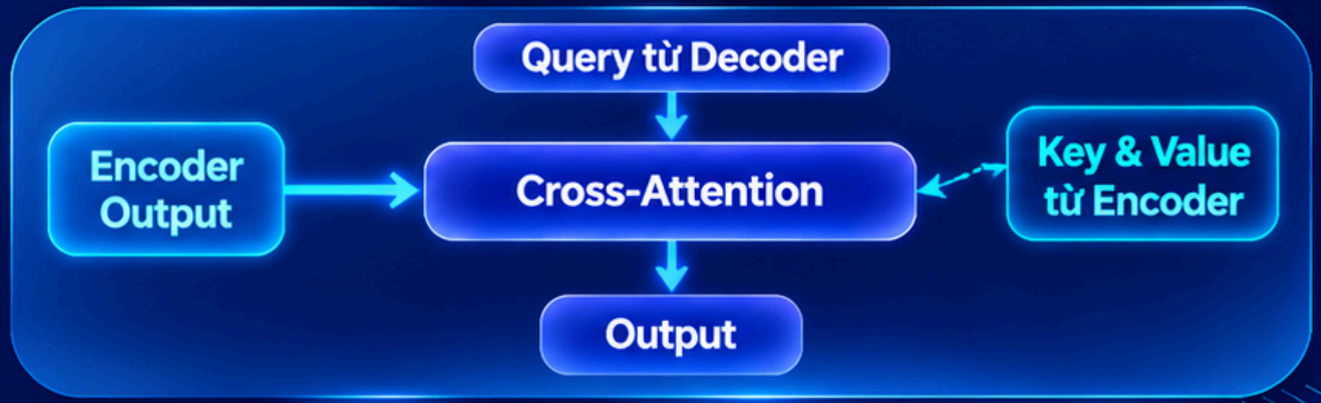
Masked Multi-Head Self-Attention

- Giống Self-Attention nhưng thêm **Mask** để ngăn nhìn thấy token tương lai.
- Giúp Decoder chỉ dùng thông tin đã có trước đó.



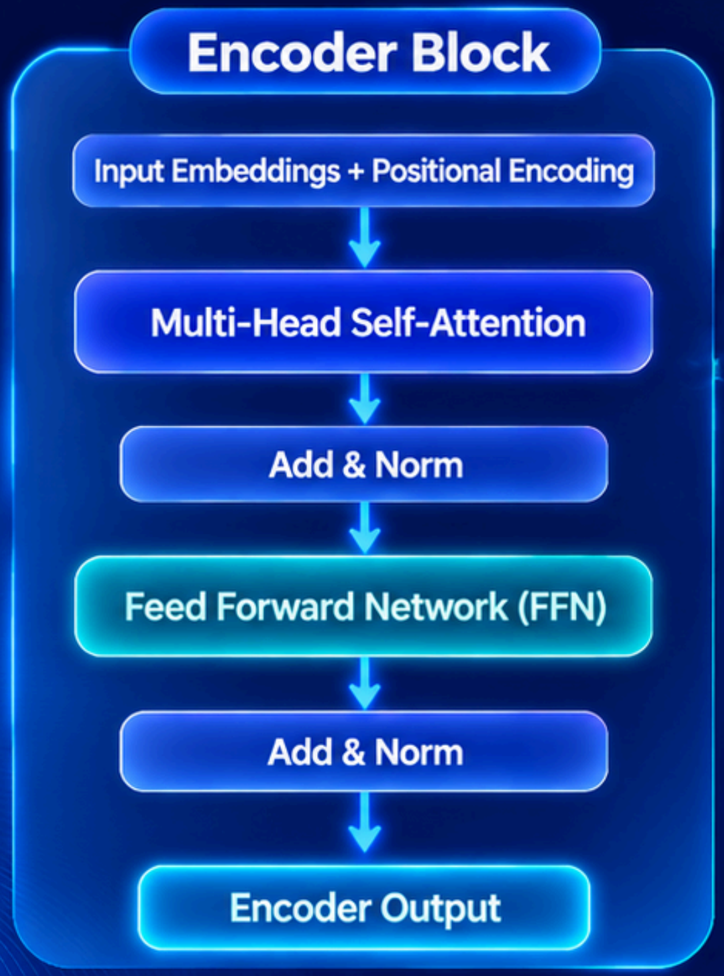
Cross-Attention

- Cho phép Decoder tham chiếu thông tin từ **Encoder Output**.
- Query từ Decoder, Key & Value từ Encoder.



→ **Masked Self-Attention giúp Decoder tự nhìn lại quá khứ, còn Cross-Attention giúp kết nối với Encoder.**

So sánh Encoder Block & Decoder Block



Tiêu chí	Encoder Block	Decoder Block
Self-Attention đầu tiên	Multi-Head Self-Attention	Masked Multi-Head Self-Attention
Attention thứ hai	Không có	Cross-Attention
FFN	Có	Có
Thứ tự chính	SA → FFN	Mask SA → Cross-Attn → FFN
Mục tiêu	Mã hóa, xây dựng biểu diễn ngữ cảnh	Sinh chuỗi đầu ra dựa trên Encoder
Đầu vào	Câu nguồn	Câu đích (dịch/ sinh)
Đầu ra	Biểu diễn ngữ cảnh	Chuỗi token đầu ra

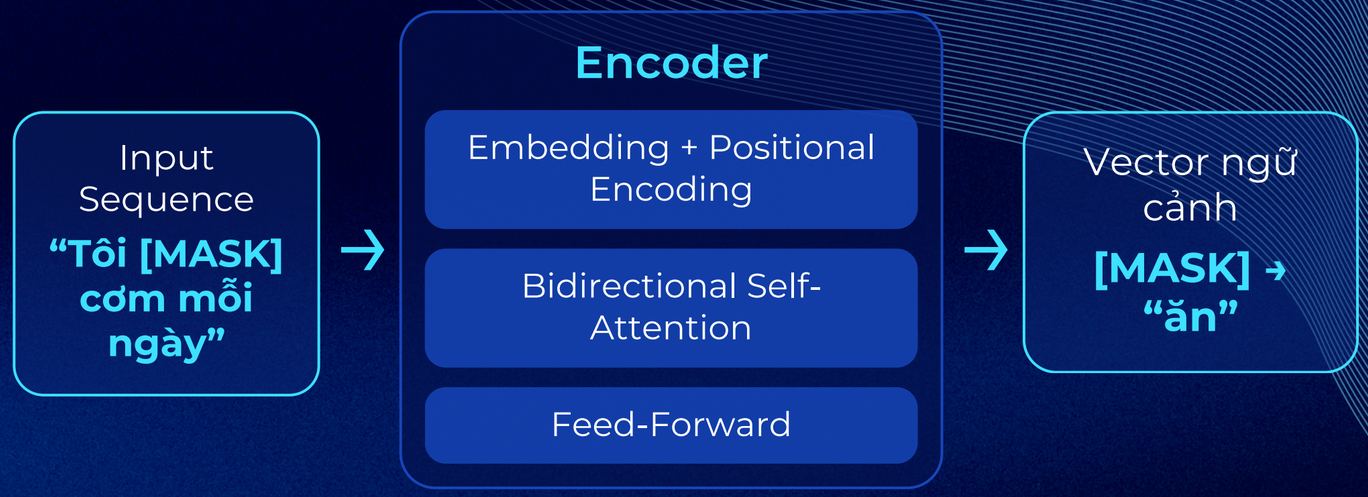
→ **Encoder tạo biểu diễn ngữ cảnh, Decoder sử dụng biểu diễn đó để sinh ra chuỗi đầu ra chính xác.**

7. CÁC DẠNG TRANSFORMER

7.1 Encoder-only

Chỉ dùng khối Encoder, mạnh về hiểu văn bản

- **Self-attention hai chiều:** mỗi token nhìn cả bên trái và bên phải.
- Đầu ra là vector ngữ cảnh cho từng token.
- Huấn luyện bằng Masked Language Modeling (che token rồi dự đoán).
- Không được thiết kế để sinh văn bản tự hồi quy từng token như GPT



[MASK] nhìn cả “Tôi” (bên trái) và “cơm mỗi ngày” (bên phải) để đoán ra “ăn”.

Tiêu biểu: BERT. Dùng cho phân loại, NER (Named Entity Recognition), QA (Question Answering)

7.2 Decoder-only

Chỉ dùng khối Decoder, mạnh về sinh văn bản

- **Masked self-attention (causal):** vị trí *t* không nhìn thấy vị trí sau *t*
- Không có cross-attention, vẫn có causal self-attention, Feed-Forward, kết nối phần dư và chuẩn hóa.
- Huấn luyện bằng dự đoán token kế tiếp (autoregressive).
- Sinh từng token một, đơn giản và dễ scale.

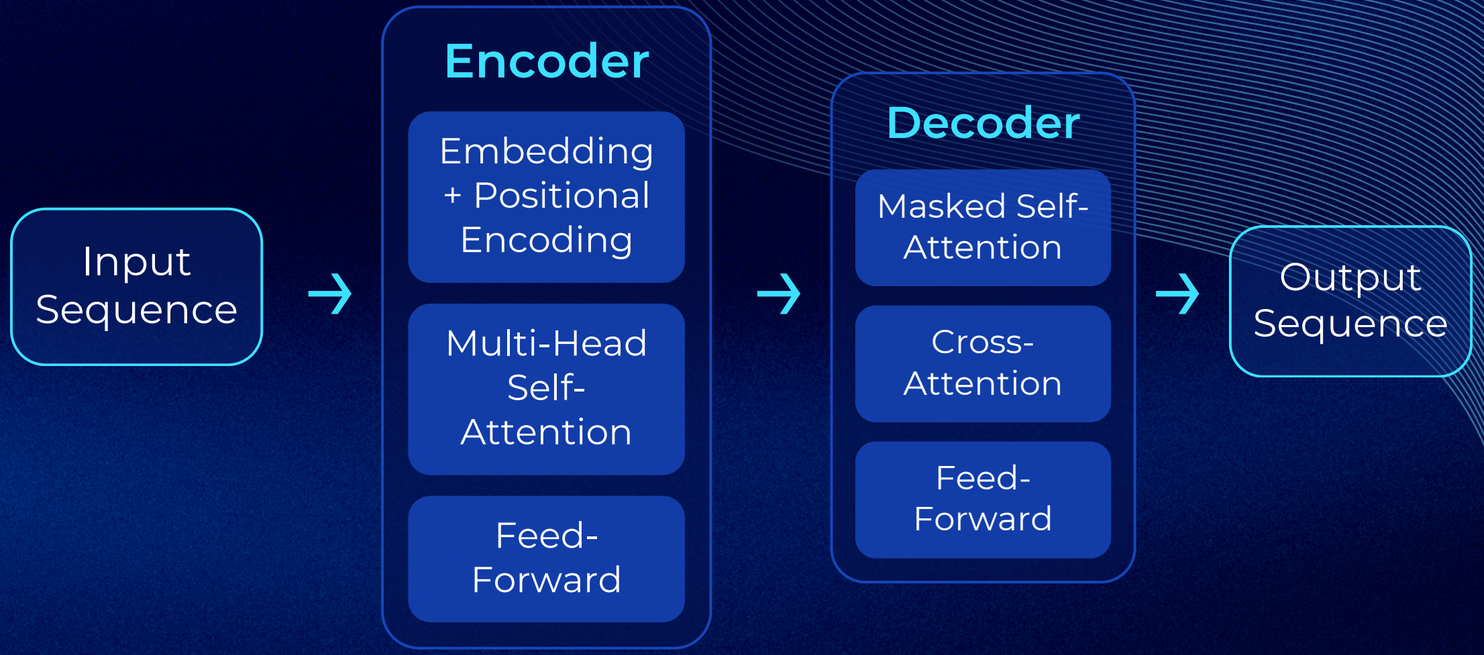
Token “Việt” được nối vào prompt, mô hình lặp lại để dự đoán token tiếp theo là “Nam”.

Tiêu biểu: GPT, LLaMA, nền của hầu hết LLM hiện nay.

7.3 Encoder–Decoder

Kết hợp cả hai khối, chuyển chuỗi nguồn thành chuỗi đích

- Encoder đọc toàn bộ input và tạo biểu diễn ngữ cảnh.
- Decoder sinh output, dùng **cross - attention** (Q từ decoder, K/V từ encoder).
- Phù hợp khi input và output là hai chuỗi khác nhau.
- Huấn luyện kiểu seq2seq (dịch, span corruption).



Ví dụ: “translate English to German: That is good.” → “Das ist gut.”

Decoder sinh từng từ và dùng cross-attention để nhìn lại toàn bộ câu input.

Tiêu biểu: T5, BART, Transformer gốc (Vaswani, 2017).

BERT, GPT, T5 - KIẾN TRÚC VÀ TÁC VỤ

So sánh kiến trúc, cơ chế attention và tác vụ

TIÊU CHÍ	BERT	GPT	T5
Dạng kiến trúc	Encoder-only	Decoder-only	Encoder–Decoder
Attention	Hai chiều	Một chiều (causal)	Encoder: hai chiều Decoder: causal
Cross-attention	Không	Không	Có, tới Encoder
Tác vụ phù hợp	Phân loại, NER, QA trích xuất	Chatbot, sinh văn bản, sinh mã lập trình	Dịch máy, tóm tắt, QA text-to-text

Cài đặt Self-Attention từ đầu

Tự triển khai phép tính attention bằng PyTorch và kiểm chứng trên trọng số GPT-2

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + M) \cdot V$$

d_model = 768 · 12 heads · d_k = 64

01

Chiếu tuyến tính

Projection

X → Q, K, V

Wqkv + bias

(T, 768)

X Q K V

02

Tách 12 đầu

Multi-Head

Reshape + transpose

Q, K, V mỗi đầu

(12, T, 64)

03

Tính độ tương đồng

Scaled Dot-Product

Q × K^T / √64

Điểm attention

(12, T, T)

04

Chặn tương lai

Causal Mask

Điền −∞ vào vị trí

không được nhìn

(12, T, T)

05

Softmax & nhân V

Attention Output

Softmax(scores)

Attention × V

Ghép → (T, 768)

Attn × V →

KIỂM CHỨNG VỚI GPT-2

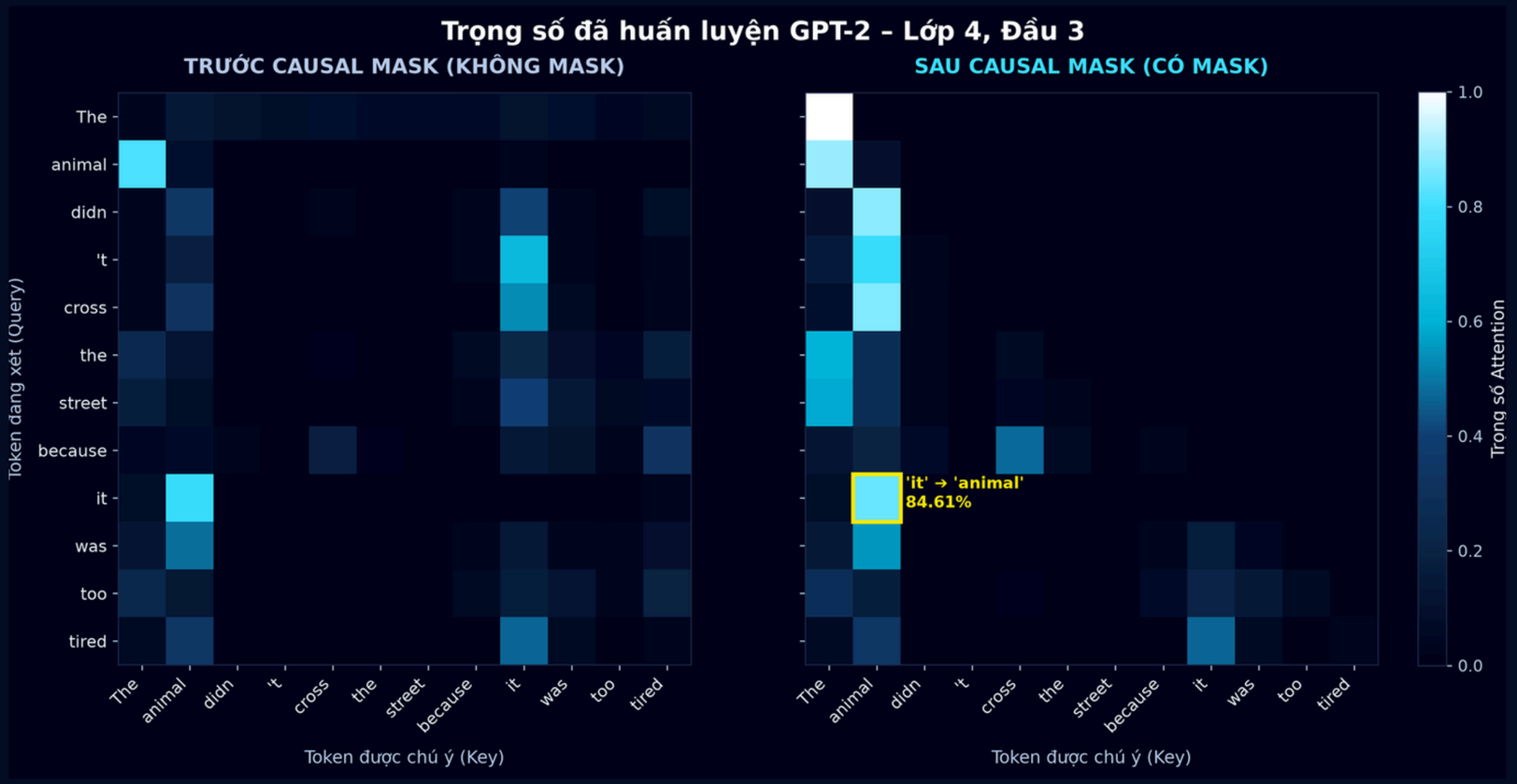
Cùng trọng số pretrained · Kiểm thử 12 layers

Sai số tối đa $1.76 \times 10^{-6} < 1.00 \times 10^{-4}$

Lưu ý: “From Scratch” là tự viết phép attention, không phải huấn luyện GPT-2 từ đầu.

Causal Mask thay đổi attention ra sao?

Trực quan hóa trước / sau mask trên cùng câu và một attention head được chọn



ĐỌC HEATMAP

01 Trước causal mask

Token có thể chú ý đến cả các vị trí ở tương lai.

02 Sau causal mask

Vùng bên phải đường chéo có trọng số bằng 0.

03 Quan sát nổi bật

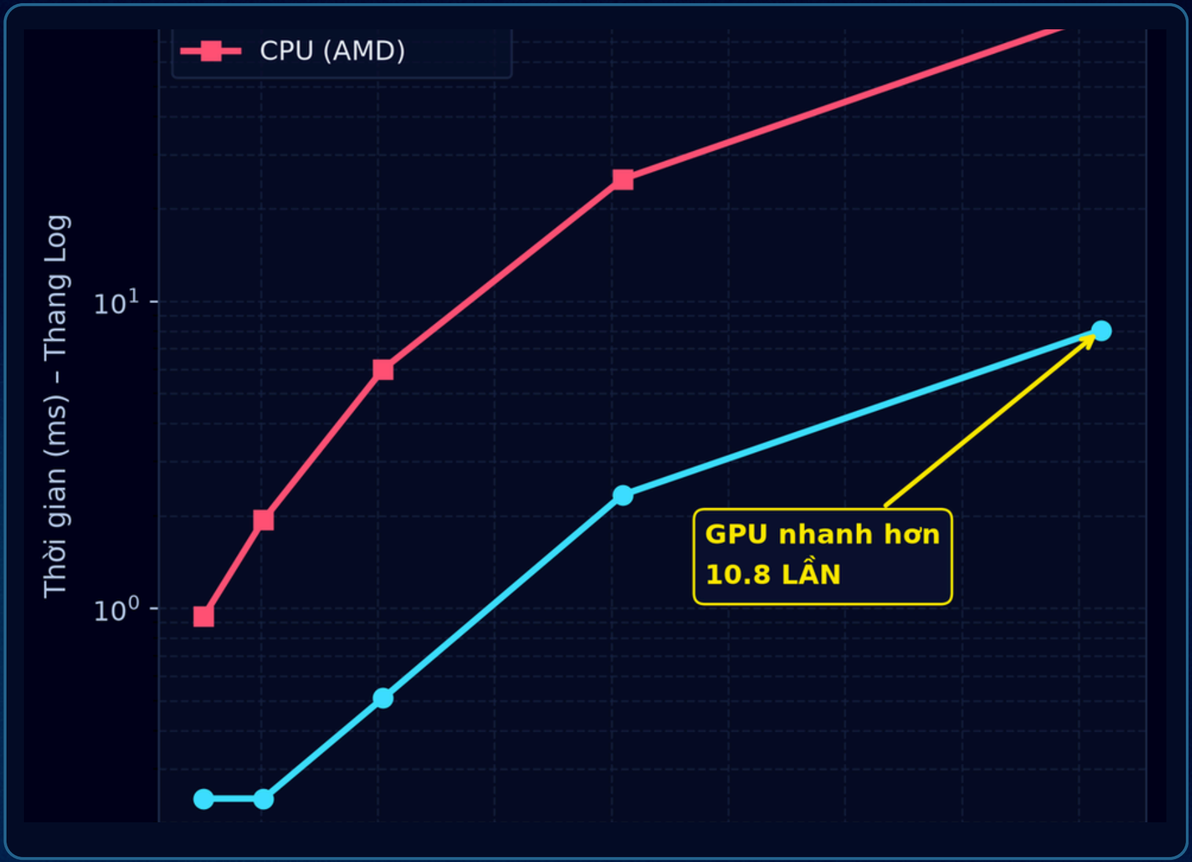
“it” → “animal”: 84,61% attention ở head minh họa.

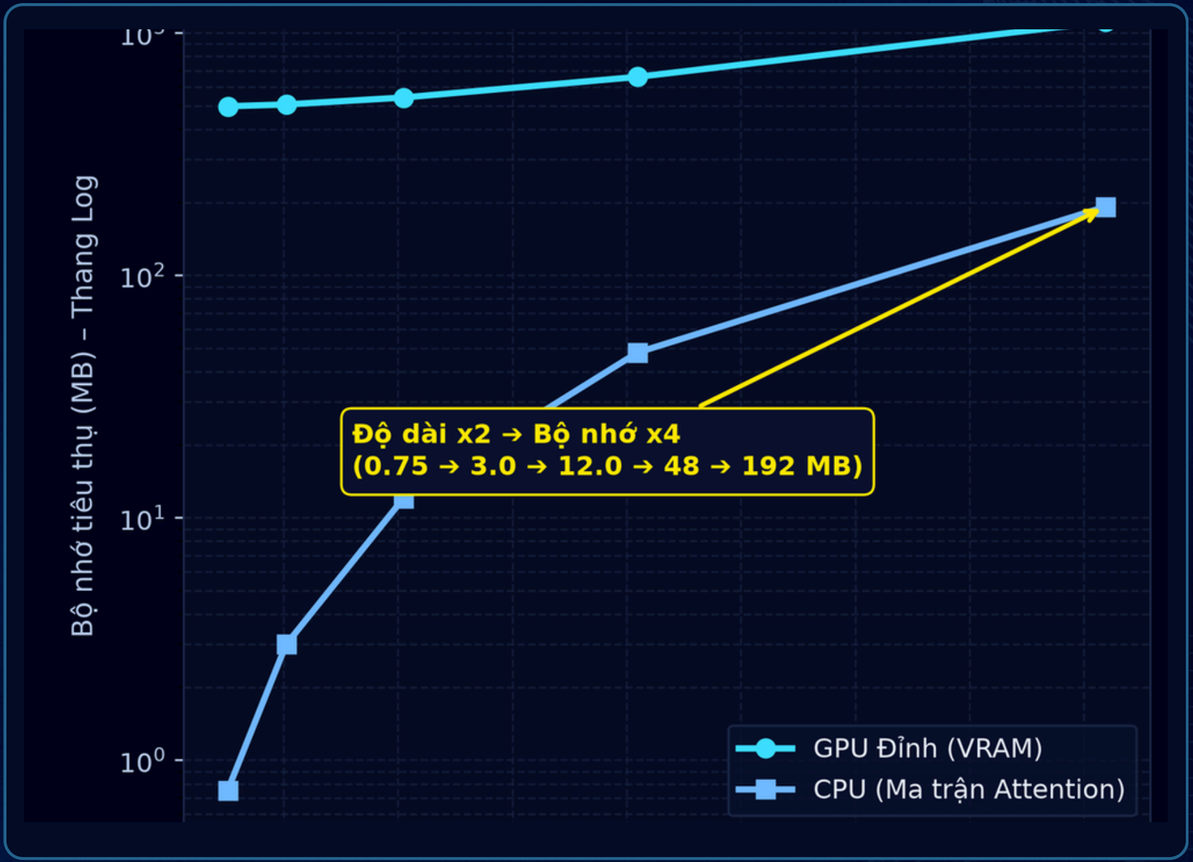
KẾT LUẬN Causal Mask ngăn mô hình “nhìn trộm” token tương lai khi sinh văn bản tuần tự.

Ví dụ: “The animal didn’t cross the street because it was too tired.” | Một head không đại diện cho toàn bộ mô hình.

GPU vs CPU: tốc độ và chi phí bộ nhớ

Benchmark với dữ liệu ngẫu nhiên · Độ dài chuỗi 128 → 2048 token · Trục tung log





GPU nhanh hơn **10,8×** tại 2048 token

Ma trận attention: **0,75 → 192 MB**

Lưu ý: kích thước ma trận attention ≠ tổng VRAM đỉnh. Các giá trị là kết quả phép đo ở cấu hình thử nghiệm.

Thank You
For Your Attention

Any Questions?